# IP2Live: 1½-Day Midterm Execution Guide

IT411 | G1–G8 | Semester 1 | AY 2026–2027

Prepared 15 October 2026 • Based on the supplied MVP findings and the local project snapshot

## 1. Your target and the first 30 minutes

**Target:** deliver one working release, updated existing documents, and verifiable QA evidence that all describe the same version. The release should be ready for later final user/stakeholder validation.

**Realistic capacity:** plan for **12 focused hours per person** across one full day and one half-day: 36 person-hours with three teammates, or 24 with two. This is feasible for consolidating an existing system and revising existing documents. It is not enough for a major redesign, a new deployment architecture, or a new effectiveness study. Use the three-person schedule below; the two-person fallback is on page 6.

**The main work is to prove and document the improvements you already have, then close a small number of critical gaps.** Do not spend this sprint adding more game modes or polishing every screen.

During the first 30 minutes:

1. Open the original SRS, SDD, SPMP, STD and RTM. Make revision copies; preserve the previous submissions and requirement IDs.
2. Assign the three roles below, create the required Drive folders, and create an evidence index and defect log.
3. Record the starting Git commit and working-tree status. The reviewed workspace contains many uncommitted changes and new files; the latest commit alone does not identify the current software.
4. Start the actual release/export and check evaluator access. Resolve the Windows-desktop versus live-URL requirement immediately.
5. Select at most **two or three small implementation fixes** from confirmed defects. Keep the remaining time for verification, documents and packaging.

| Role | Primary responsibility | Required handoff |
|---|---|---|
| A — Developer / release owner | Critical fixes, export/deployment, technical explanations, retests | Working release; commit IDs; build instructions; limitations |
| B — Documentation / submission owner | SRS, SDD, SPMP, RTM, README, feature matrix, Drive organization | Consistent documents and working evidence links |
| C — QA / evidence owner | STD, test execution, performance, formative usability, defect log | Raw logs; actual outcomes; retests; acceptance summary |

**First decision gate — by Day 1, 09:00:** establish how evaluators will access a runnable system. A desktop download URL is not automatically the live system URL requested in the brief. If no existing web build works, obtain the instructor's decision on desktop distribution; do not assume an exception. Keep working on independent documents and QA while that decision is pending.

**How to read this guide:** "present in source" is an inspection finding, not a runtime pass. "Not found locally" means the reviewed repository did not contain the evidence; your original documents and Drive may already contain it. No tests or deployment checks were executed as part of preparing this guide.

# 2. MVP improvement audit: what exists and what still needs work

The MVP findings below are taken from your supplied summary. The original survey data and previous formal documents were not available for comparison. Treat the following as a targeted audit checklist, then reconcile it with those originals.

| MVP finding | What the current project contains | Remaining action for this midterm |
|---|---|---|
| Stage 3–4 difficulty jump; Stage 4 engagement 3.61 and 12/31 students reporting more than five retries | CIDR and VLSM tutorials; optional practice catalog includes VLSM; revised Gameplay 5 instruction | Verify a learner can reach and use the support through the normal flow. Confirm the intended untimed-practice behavior. Record a guided-to-independent Stage 4 walkthrough. Progressive hint escalation and a complete bridge sequence are not established by this audit. |
| Diagnostic Rewind needs more specific explanations | Diagnostic screen and tutorial replay infrastructure; VLSM validation displays a reason and hint | Check mask, prefix, capacity, boundary and overlap errors individually. Verify useful feedback appears at the right moment. Do not equate a generic replay offer with an explanation of the actual error. |
| Information Quality was the weakest usability area, 1.97 | Tutorial wording, dialogue typography and layout tests; Gameplay 5 lesson documentation and screenshots | Inspect clarity at a smaller resolution and normal resolution. Check repeated tutorials and recovery after mistakes. Capture actual observations and any small wording/layout corrections. |
| Need exact attempts, retries, answers, timing and help use | Assessment schema v2, decision histories, session/attempt identity, timing, tutorial/recovery events, durable reports and learning analytics | Verify the requested field set across representative activities, especially VLSM. Check that retry counts have an explicit meaning. Export a real test session and reconcile it with what the tester actually did. |
| Camera, pathfinding, progression and save/load defects | Save lifecycle, autosave, resume, input/quest recovery and progression tests; desktop storage/release tools | Reproduce the original reported defects where steps exist. Verify in the packaged game, not only a test harness. Camera/pathfinding stability remains unverified by this inspection. |
| Users trusted answer checking, 4.38/5 | Shared CIDR helpers and VLSM capacity/alignment/overlap checks | Independently verify mathematical outputs and invalid-case rejection. Trust ratings do not prove validator correctness. Dedicated VLSM acceptance coverage needs checking. |
| Perceived competency rose 3.59 → 3.97 | Reports explicitly distinguish descriptive in-game performance from validated learning | Preserve that distinction in SRS, SDD, STD and discussion. Do not claim proven learning, causation or transfer from the survey or telemetry. |
| Strong engagement and educational-value ratings | Existing Story, Practice, Endless and PVP implementation/documentation | Document the modes included in the chosen release and their limitations. Prioritize the learning bottleneck and reliability over adding modes. |

**Important scope limit:** the Combined toolkit is enabled only at Stage 4 Level 4 for testing. Its documentation explicitly defers final-puzzle integration; its tools do not complete Story quests or write assessment reports. Mark this **partial / experimental** if included. It is not evidence that the final Stage 4 learning activity is complete.

# 3. Submission readiness: the gaps to close first

| Deliverable | Finding in the reviewed workspace | Action and priority |
|---|---|---|
| Live deployment URL | Windows desktop/Electron release tooling is present; no evaluator-ready live URL was identified in the reviewed README/deployment material | **P0:** resolve delivery route and instructor acceptance early; test the agreed route from another device |
| Source repository | Local origin points to https://github.com/res-jammixs/IP2Live | Confirm evaluator access and submit the exact release commit/tag; remote availability was not checked |
| README and setup | README describes features but lacks complete setup, release, access and deployment instructions | **P0:** add reproducible editor/export steps, prerequisites, launch instructions, URLs and limitations |
| SRS, SDD, SPMP, STD and RTM | No formal versions were identified locally; you confirmed originals are available elsewhere | **P0:** revise the originals and bring them into the evidence package; feature notes do not replace these documents |
| Revision histories | Formal documents were unavailable for inspection | Add a revision history to **each of SRS, SDD, SPMP and STD**, with reason and affected sections |
| Feature completion matrix | No consolidated submission matrix identified | **P0:** label implemented, partial, deferred and not verified separately |
| Testing evidence | 75 automated test files were identified, plus screenshots and feature notes | **P0:** capture execution logs on the release candidate and actual manual results; test code and prose claims are not execution records |
| Performance report | Pause rendering test uses mocked Canvas behavior; no consolidated measured runtime report identified | **P0:** measure actual workloads and report response times, throughput, error rate and environment |
| Usability and defects | Initial survey summary supplied; old bugfix verification note exists | Retain baseline findings; add current formative observations, issue status and retests. Reconcile old issues instead of calling them fixed without a current check |
| Acceptance results and RTM evidence links | No consolidated release acceptance table identified | Link requirements to cases, actual results, defects and evidence; show untested items explicitly |

**Resolve in this order:** access/deployment risk → SRS and RTM baseline → STD and critical tests → small fixes and retests → SDD/README consistency → final evidence package. The SPMP receives an immediate roles/schedule/risk update and a final status update.

**Limit coding to confirmed problems.** Examples of acceptable small fixes: wrong validation result, missing error explanation, broken progression, duplicate tutorial trigger or lost save state. A complete Stage 4 redesign, new analytics architecture or browser port is outside this sprint's realistic capacity.

**Protect the submission scope:** if an original requirement cannot be met, record it as unmet/deferred with its impact. Do not remove it from the SRS simply to improve the pass rate. A feature outside the MVP priorities may be deferred only if the original requirements and instructor expectations allow it.

# 4. Which documents to prioritize, and exactly what to revise

**Start with SRS + RTM together.** They establish what the release must do and how you will prove it. Draft the STD next, before recording test outcomes. Revise the SDD alongside implementation; update SPMP briefly at kickoff, then reconcile every document after testing.

| Document | Priority and first drafting budget | Required updates |
|---|---|---|
| SRS | First; about 75 minutes using the original | Preserve IDs; revise Stage 3–4 support, error feedback, telemetry fields, save/reload, readability and supported platform. State measurable acceptance conditions, operating constraints and exclusions. Separate functional behavior from educational-effectiveness claims. |
| RTM | Start with SRS; about 30 minutes | Map MVP finding → requirement → design/module → test → actual result → evidence/defect. Seed it immediately and fill results during QA. |
| STD | Next; about 60–90 minutes | Document environment, cases, steps, inputs, expected outcomes, coverage, entry/exit conditions and performance workload. Keep planned tests separate from actual execution records. |
| SDD | Parallel with fixes; about 90–120 minutes | Update actual architecture, mode/progression flow, tutorial/recovery behavior, validator rules, telemetry model, save transaction and deployment. Show Windows-only services versus browser fallback behavior accurately. |
| SPMP | 20–30 minutes at kickoff; 30 minutes to finalize | Replace outdated tasks with this sprint, actual owners, hours, dependencies, scope, risks, configuration control and the later final-validation milestone. Record unresolved deployment or quality risks. |
| README + feature/refactor evidence | Start access instructions early; allow 60–90 minutes overall | Exact prerequisites and setup; project opening/export; release preparation; launch/use instructions; test commands; deployment URL; repository release; data behavior; limitations; meaningful before/after changes. |

These are **first-pass drafting budgets**, not total effort. Reserve another two to three hours of the documentation owner's day for cross-checks, evidence links, revisions and uploading.

**Revision history: insert into each of the first four documents.** Use your document's next existing version number; do not label the old version as newly tested.

| Date / version | Section or requirement IDs | Change | Reason / MVP finding | Author / reviewer |
|---|---|---|---|---|
| [Fill when revised] | [Affected sections] | [Concrete change] | [Finding or defect ID] | [Actual names] |

Also retain the prior version and a short consolidated change summary. For an implementation already completed before this sprint, say so and cite its real commit; do not imply it was developed during these 1½ days.

**Consistency check:** every document should agree on platforms, modes, Stage 4 status, save behavior, telemetry meaning, assistance rules and release ID. Distinguish campaign stage/level from gameplay number: Gameplay 8 is the VLSM module, not "Stage 8."

# 5. Day 1: eight focused working hours

Times are an example local schedule; shift the blocks to your actual start date. Day 1 runs 08:00–18:00 with two hours of lunch/breaks. Work in parallel by role, with a five-minute handoff at each gate.

| Time | A — Developer / release | B — Documents / submission | C — QA / evidence | Required output |
|---|---|---|---|---|
| 08:00–08:30 | Start release/export; record current state | Open originals; assign owners; create Drive structure | Create evidence index and defect log | Starting version, owners, scope shortlist |
| 08:30–09:30 | Prove access route; inspect critical defects | Revise SRS and seed RTM; update SPMP roles/risks | Inspect existing tests; draft acceptance conditions with B | **09:00 access gate; 09:30 requirement baseline** |
| 09:30–10:30 | Reproduce highest-risk defects; select fixes | Finish SRS baseline; draft actual SDD changes | Finish STD critical cases; execute existing suite and preserve logs | Test-ready baseline and confirmed defect list |
| 10:30–10:45 | Break | Break | Break | — |
| 10:45–12:15 | Implement first small fix; help verify validator | Revise SDD and revision histories | Run functional/math checks; retest first fix | One reviewable fix plus linked result |
| 12:15–13:15 | Lunch | Lunch | Lunch | — |
| 13:15–14:45 | Finish only selected critical fixes; prepare candidate | README, feature matrix and refactor evidence | Save/load, progression and telemetry integration tests | **14:45 feature freeze**; no further scope additions |
| 14:45–15:00 | Break | Break | Break | — |
| 15:00–16:30 | Execute packaged performance workload; support runtime checks | Finalize SDD draft; update RTM with observed status | Conduct short formative usability checks; compile performance results | Raw metrics; observed usability issues; actual failures |
| 16:30–17:00 | Break | Break | Break | — |
| 17:00–18:00 | Fix a blocker only if a retest fits Day 2 | Reconcile documents and known limitations | Finish initial QA summary; identify required reruns | Candidate, remaining defects and Day 2 task list |

**Day 1 exit:** the access route is established or explicitly unresolved; all five documents have updated drafts; automated logs and priority manual results exist; at most one or two small issues remain for retest. Do not leave performance collection or all formal documentation until the last morning.

**If behind at 14:45:** stop enhancements. Prefer verifying an existing tutorial or validator over building a new one. Escalate an unmet must-have requirement honestly rather than claiming completion from screenshots.

# 6. Day 2: four focused working hours and the two-person fallback

| Time | A — Developer / release | B — Documents / submission | C — QA / evidence | Required output |
|---|---|---|---|---|
| 08:00–09:00 | Complete blocker fix only; review intended release files | Finish SRS/SDD/SPMP revisions and change summary | Retest defects; run relevant regression checks | **09:00 final source freeze** |
| 09:00–10:00 | Commit reviewed release; rebuild/export; publish agreed delivery | Record full commit, release ID and artifact checksum | Smoke-test exact artifact; rerun affected measurements/tests | Matching source, package and QA results |
| 10:00–10:15 | Break | Break | Break | — |
| 10:15–11:15 | Confirm access from a second device; resolve release issues | Finish README, feature matrix, RTM and evidence links | Finalize performance, defect, usability and acceptance reports | Complete submission folders |
| 11:15–12:15 | Check release and repository links | Upload final files; audit permissions and naming | Independently open evidence; sign off actual results | Submission index and final go/no-go |

If the release was unchanged since Day 1, reusable evidence must identify that same source state and environment. If code changed, rerun affected tests and relevant regression checks. A change to the validator, timing, save path or package requires the corresponding performance/integration checks again.

## If only two people are available

Use A as developer/release owner and B as documentation/QA owner. A must also execute approximately four hours of QA; B must alternate document revisions with evidence capture. This leaves roughly **3–4 hours for coding**, so allow **one critical fix** and reuse the existing document structure and automated suite.

Keep every required deliverable. Reduce optional mode exploration, extra screenshots and cosmetic work. Do not omit performance, actual results, revision histories or the RTM. Reuse prior usability evidence as the baseline, label its date, and add a brief current review rather than organizing a new survey.

If deployment needs a port, several critical features are still broken, or the originals require reconstruction, **full completion in 24 person-hours is unlikely**. Report the remaining requirements and request a scope/deadline decision; a smaller test sample does not establish broad coverage.

## Hard gates

- **09:30 Day 1:** requirements and test expectations are explicit.
- **14:45 Day 1:** enhancements stop; only selected defects can change.
- **09:00 Day 2:** final source is frozen and uniquely identified.
- **11:15 Day 2:** every required item has a real file/link or a clearly recorded gap.
- **12:15 Day 2:** evaluator access and release consistency have been checked independently.

# 7. Translate findings into requirements and the feature matrix

Use the examples below to revise your **existing IDs**. These are proposed acceptance conditions to confirm against the original scope, not newly verified features. Do not silently replace stronger existing requirements.

| Example requirement | Suggested acceptance condition | Evidence to collect |
|---|---|---|
| REQ-SUPPORT — Stage 3–4 scaffolding | Learner can access relevant CIDR/VLSM instruction or practice, then attempt the independent activity without corrupting campaign progress | Normal-flow recording; test steps; saved state before/after |
| REQ-FEEDBACK — Specific correction | Supported invalid mask/prefix/capacity/boundary/overlap answers receive feedback that identifies the observed error and a useful next step | Invalid inputs, actual messages and retry result |
| REQ-VALIDATOR — Correct subnet checks | Known valid outputs are accepted and invalid capacity/alignment/overlap/containment cases rejected according to documented rules | Independent expected-answer table and execution record |
| REQ-TRACK — Traceable activity | A known test session retains stage/topic, answer, correctness, error category, attempt/retry identity, timing, help/rewind use, completion and session identity where required | Event JSON plus reconciliation to tester actions |
| REQ-SAVE — Reliable resume | Manual save, restart and resume restore the documented stage, position and puzzle state; new profiles do not inherit another learner's state | Packaged-game steps and before/after captures |
| REQ-INFO — Clear instructions | Text and controls remain usable at supported resolutions; repeated tutorials and recovery follow documented rules | Formative task notes; screenshots; defect/retest |

**RTM row format:** finding ID | requirement ID | design section/module | test ID | result | defect ID | evidence link | release ID. Use Pass / Fail / Blocked / Not run; blank evidence is not a pass.

**Feature completion matrix format:** feature | original requirement(s) | implementation state | verification state | limitation | evidence | owner.

Seed the matrix with: Story/progression; Stage 3–4 instruction; Practice; Diagnostic Rewind; subnet/VLSM validation; telemetry/export; save/load; deployment; and optional Endless/PVP/Combined if they are included in the release. Keep implementation state separate from verification state.

**Known limitations to reconcile:** Combined final-puzzle integration is deferred; LAN PVP depends on the desktop bridge/network setup; Endless unlocks after Story completion; missing historical telemetry cannot be reconstructed; learning indicators are descriptive. Confirm which apply to your chosen artifact rather than copying every feature claim into the SRS.

**Exit rule:** all must-have requirements need actual acceptance results. Any failed, blocked or untested must-have remains a readiness gap even if most other tests pass.

# 8. Minimum test pack and automated evidence

Execute cases against the release candidate and record inputs, expected result, actual result, status, tester, date, environment and evidence path. Expand this pack for any original must-have requirement not covered here.

| Case group | Minimum checks | Test category |
|---|---|---|
| FT-01 Access and launch | Evaluator can obtain/open the agreed release; controls and assets load; relevant offline behavior works | Functional / system |
| FT-02 Normal progression | Complete representative normal Stage 1–4 transitions; prioritize Stage 3→4 and the VLSM objective; no developer bypass for acceptance | Functional / system |
| FT-03 Feedback and replay | Wrong mask, prefix, host capacity, boundary and overlap; retry; repeated-failure offer; continue; replay; no accidental loop | Functional / regression |
| FT-04 Instruction and practice | Guided→independent flow; VLSM practice access; intended timer behavior; return to campaign without changing its state | Functional / integration |
| FT-05 Save/reload | Mid-puzzle save, close/reopen, resume; repeated saves; autosave behavior; profile isolation; backup recovery if required | Integration / regression |
| FT-06 Telemetry and export | Known wrong/correct attempts, one retry, help/replay, pause/resume and completion; verify JSON/report agreement and profile ownership | Integration / system |
| FT-07 Movement and UI | Camera/rotation, pathfinding/interaction, keyboard/mouse, repeated tutorials, supported resolutions | System / regression |
| FT-08 Optional modes | Smoke-test each included mode; document unlocks, platform limits and deferred features | Functional / regression |
| AT-01 Acceptance | Match every must-have requirement to executed cases and unresolved defects | Internal requirements-based acceptance |

A full campaign playthrough may exceed the testing slot. Start from legitimate saved checkpoints to verify later-stage flows; state that limitation. Developer testing previews are useful for diagnosis, but do not prove normal campaign completion.

## Capture the existing suite from PowerShell

Run from the repository root. Change the output folder to the required QA folder if it is elsewhere. Keep the raw output, exit code, Node version and exact source state.

```
$qaFolder = 'midterm-evidence\03_Testing_and_QA\01_Automated'
New-Item -ItemType Directory -Path $qaFolder -Force | Out-Null
git rev-parse HEAD | Set-Content "$qaFolder\source-commit.txt"
git status --short | Set-Content "$qaFolder\working-tree.txt"
node --version | Set-Content "$qaFolder\node-version.txt"
$testFiles = @(Get-ChildItem -LiteralPath tests -File |
  Where-Object { $_.Name -match '\.test\.(cjs|mjs)$' } |
  Sort-Object Name | ForEach-Object { $_.FullName })
& node --test @testFiles 2>&1 | Tee-Object "$qaFolder\test-run.log"
$testExitCode = $LASTEXITCODE
"Exit code: $testExitCode" | Set-Content "$qaFolder\exit-code.txt"
```

Record failures as defects; rerun failed/affected cases after fixes. Preserve both initial and retest logs. If the source changes, produce final-state logs. A harness test can pass while the packaged game fails; retain manual runtime evidence too.

# 9. Validator checks: prove the answers independently

Use hand-checked expectations reviewed by a second teammate. Do not calculate expected results by calling the same helper being tested. The following are fixed arithmetic examples, not records of executed tests.

| Input / condition | Expected result |
|---|---|
| 192.168.10.70/26 | Mask 255.255.255.192; network 192.168.10.64; broadcast 192.168.10.127; ordinary usable range .65–.126; 62 hosts |
| Ordinary subnet must support 30 hosts | /27 provides 32 addresses and 30 usable hosts; /28 is insufficient |
| Divide a /24 into at least six equal subnets | Borrow three bits: /27, eight subnets, 30 ordinary usable hosts each |
| Submitted network address 192.168.10.70/26 | Reject as a network boundary if the field requires an aligned network; .64/26 is aligned |
| Two allocated ranges intersect | Reject overlap; adjacent, non-overlapping valid blocks may be accepted |
| Allocation lies outside the parent block | Reject containment violation |

**Current VLSM scenario in source:** parent 172.30.32.0/20; branch demands 450, 200, 100, 60 and 25 hosts. One valid largest-first allocation is:

| Branch demand | Example allocation | Ordinary usable capacity |
|---|---|---|
| 450 | 172.30.32.0/23 | 510 |
| 200 | 172.30.34.0/24 | 254 |
| 100 | 172.30.35.0/25 | 126 |
| 60 | 172.30.35.128/26 | 62 |
| 25 | 172.30.35.192/27 | 30 |

All five blocks fit inside the parent and do not overlap. Other allocations can also be valid; verify against the documented game rules rather than assuming this is the only accepted solution.

Add wrong-capacity, misalignment, overlap and out-of-parent cases. For each, retain the actual rejected input, actual message, grading result and telemetry category. Verify that the final complete route table is accepted and the intended quest progresses.

**Boundary policy:** document what the application supports for /31, /32, invalid prefixes, malformed addresses and classful-default versus assigned CIDR masks. Test the declared policy; do not claim support for an edge case merely because the helper returns a value.

**Practical output:** one validator-results table plus a small set of screenshots/log excerpts. If validator correctness fails, it takes priority over cosmetic changes or optional modes.

# 10. Performance: actual measurements in a bounded workload

Reserve about **60–75 minutes** for measurement and reporting. Use the actual exported/deployed artifact, a dedicated test profile and a documented dataset. Record device/CPU/RAM, OS, resolution, engine/runtime, storage, network conditions, release ID and measurement method. Confirm existing SRS performance thresholds before execution; if none exist, report descriptive results without inventing a pass threshold afterward.

| Workload | Suggested repetitions | Start → stop boundary |
|---|---|---|
| Cold application launch | 3 | Process launch → interactive main menu |
| Answer validation | 10 | Submit → feedback visible, on a fixed documented task |
| VLSM validation | 10 | Commit/check → result visible; include valid and invalid allocations |
| Save | 10 | Save request → durable success acknowledgement |
| Load/resume | 10 | Load request → restored playable scene |
| Report export | 5 | Export request → usable PDF/spreadsheet/evidence package |

This is a one-client workload of 48 operations, not a concurrent-load benchmark. Record profile, save size, event/attempt count and report scope. Use an existing test dataset; if a synthetic dataset is necessary, identify it clearly and exclude it from student learning claims. Separate first-run initialization from warmed operations.

Capture each observation using logs/profiling where available. For visible response times, use a screen recording or timestamped instrumentation with a stated precision; a coarse stopwatch does not support millisecond claims. Never show a save as successful before its durable acknowledgement merely to improve timing.

**Raw CSV fields:** run_id, operation, release_id, workload_size, start_time, end_time, duration_ms, success, error, environment_id. Keep raw records and any measurement script/recording.

**Summary per operation:** sample count; mean; median; p95; maximum; successes/failures; error rate. Use p95 as the sorted observation at position ceil(0.95 × n), and label the small sample. Throughput = successful operations / measured workload duration in seconds; state whether the interval includes pauses/think time. Do not present inverse latency as measured throughput.

For a real web deployment, also record actual network response times, request throughput and HTTP error rate under the tested workload. Desktop operations/second do not establish server capacity. If no server exists, mark server-concurrency metrics **not applicable** and report relevant client metrics; this does not resolve the live-URL requirement.

**Performance report sections:** objective/thresholds; environment; workload; method; raw-data link; results table; observed bottlenecks; limitations; fix/retest comparison if applicable. Add FPS/frame time and peak memory if measured. The mocked pause-rendering regression test is supplementary evidence, not the runtime performance report.

# 11. Usability, defects and internal acceptance evidence

## Keep the initial validation and current QA distinct

Use your supplied survey as **MVP baseline evidence**, with its original date, sample and instrument. Preserve the reported Stage 4 retries, Information Quality and stability findings. Do not relabel those scores as post-refactor results.

During this sprint, run a short **formative usability review** with two or three available peer testers if feasible. Give each the same tasks: locate VLSM help/practice; recover from an invalid allocation; save/resume; find/export the report. Record task success, observed confusion, help requests, approximate completion time and verbatim short comments without personal identifiers.

Use about 10–15 minutes per tester, then make only small justified corrections and retest the affected task. Report the actual sample and selection. If only teammates can participate, label the activity an internal walkthrough; do not claim independent-user usability testing. If no current participants are available, state that current usability results are missing and retain the MVP evidence as baseline only.

**Usability record:** tester code | build | task | observed outcome | issue | change | retest result | evidence. Two or three observations cannot establish an improvement in the original survey score or reduced Stage 4 retries for the student population.

## Defect log and retest

| Required field | What to record |
|---|---|
| Identification | Defect ID, linked finding/requirement, title, severity, owner |
| Reproduction | Environment/build, steps, input, expected and actual outcome, initial evidence |
| Resolution | Status, actual fix commit or reason deferred, impact/workaround |
| Retest | Date, tester, exact tested build, outcome, retest evidence, related regression result |

Use Open / In progress / Fixed awaiting retest / Closed after pass / Deferred / Not reproduced. A proposed fix is not a closed defect. Keep the initial failing evidence even after the retest passes. Compare the old bugfix note with current behavior; it contains partial/correction findings, not blanket proof of current reliability.

## Acceptance summary

Run **internal requirements-based acceptance** against the SRS/RTM. State total in-scope requirements, those passed, failed, blocked and not tested; list open critical defects and any deployment exception awaiting approval. Readiness requires no unresolved critical blocker and evidence for every must-have requirement.

This is preparation for later final user/stakeholder validation. Do not mark final stakeholder sign-off or educational effectiveness as completed. Any unexecuted case remains Not run, even if a similar automated test exists.

# 12. Deployment, refactoring and version alignment

## Establish a repeatable release

1. Confirm the chosen platform and evaluator access route at kickoff. Check an existing web export if one is already available; do not begin a desktop-to-web rewrite during this sprint.
2. For Windows, document the exact RPG Paper Maker version, editor/open-project steps and export settings. Review the existing `tools/prepare-windows-release.ps1` parameters: SourceExportPath, DestinationPath, ProjectRoot and ReleaseVersion. Prepare into a new release directory; preserve source files and existing learner saves.
3. Review all changed/untracked release files with the team. Include only intended files; do not use an indiscriminate commit of the entire workspace. Avoid shipping personal Saves, Backups, learner data, credentials or generated local logs.
4. Create a real release commit and record its full hash. Build/export from that source, apply the documented preparation process, and record the artifact checksum. The earlier inspected HEAD was `1362354`; it is not automatically the final release.
5. Execute final smoke/retest checks on the exact artifact. Record source commit, release version, artifact filename/hash, test date/environment and delivery URL in one release manifest.
6. Open repository, software and evidence links from another device/account. A teammate already signed in with owner access is not a valid evaluator-access check.

**Desktop access:** provide supported Windows version, download/extract/run steps, controls, save location, profile setup and offline limitations. If no authentication is required, say so. Use a fresh evaluation profile or a sanitized checkpoint where permitted. Explain legitimate unlock steps; developer shortcuts must be labeled.

**Live access:** verify the system itself runs at the supplied URL. A video, screenshot gallery or download page is supporting evidence and does not independently satisfy a requirement for a live running application. Keep any instructor-approved alternative in writing.

## Refactoring evidence that a reviewer can inspect

Choose two or three meaningful changes and link their real commits/PRs or reviewed diffs. Describe the prior problem, resulting behavior, affected module and verifying test. Good candidates to inspect are shared gameplay registration, tutorial lifecycle centralization, durable save/report handling, and active-time/decision telemetry. Confirm which are actually committed and included in the release.

Separate a refactor that preserves behavior from a feature or defect fix. Do not manufacture commit history. A newly committed existing change should describe the real work and its verification honestly.

**Release manifest fields:** project/course; release ID; full commit; working-tree status; artifact/hash; deployment/access URL; GitHub URL; build/export procedure; supported environment; evidence root; tester/date; known limitations. Reference this manifest from all reports and documents.

# 13. Drive package, final checklist and evidence sources

Use the required master-folder names. The extra files/subfolders below make the package easy to verify; the execution guide itself is a planning aid, not a substitute for the required documents.

```
IP2Live_IT411_Midterm_[ReleaseID]/
  00_Submission_Index.pdf
  01_Software_and_Deployment/
    Links_and_Access_Instructions.pdf
    README.pdf
    Release_Manifest.pdf
    Deployment_Evidence/
    Refactoring_Evidence.pdf
    Feature_Completion_Matrix.xlsx
  02_Updated_Documentation/
    SRS_[Version].pdf
    SDD_[Version].pdf
    SPMP_[Version].pdf
    STD_[Version].pdf
    RTM_[Version].xlsx
    Consolidated_Change_Summary.pdf
  03_Testing_and_QA/
    01_Automated/
    02_Functional_Integration_System_Regression/
    03_Performance/          [report + raw data]
    04_Usability/            [baseline + current review]
    05_Defects_and_Retests/
    06_Internal_Acceptance/
```

Keep editable originals alongside final PDFs where useful. Put revision histories inside SRS, SDD, SPMP and STD as well as the consolidated summary. The index should link every required item, state the release ID and identify unresolved gaps.

**Before submission:** verify runnable access; GitHub access and matching commit; all five revised documents; four revision histories; setup/deployment information; feature matrix; meaningful refactoring evidence; actual test results; performance raw data and summary; usability evidence with honest scope; defects/retests; acceptance results; complete RTM; evaluator permissions; no confidential credentials or identifiable learner data.

**Final handoff:** A checks software and source, B checks document consistency and links, C checks the evidence behind every pass claim. Any missing required item is named in the submission index, not hidden behind a generic “completed” status.

## Basis of this audit

Reviewed locally on 15 October 2026: supplied MVP summary (its last paragraph is truncated); README; Git status/recent history; 75 test files by inventory; feature/design notes; Windows release tools; selected implementation and test sources. Your original formal documents, original survey dataset, remote repository access, Drive permissions and deployed software were not inspected. No tests were executed for this planning audit.

Key local references for the document owner: `docs/gameplay5-tutorial.md`; `docs/gameplay-assessment-telemetry.md`; `docs/learning-report-telemetry.md`; `docs/learning-improvement-metrics.md`; `docs/practice-endless-telemetry.md`; `docs/combined-gameplay.md`; `docs/gameplay-registration.md`; `docs/IP2Live_Offline_Save_Report_Music_Reliability_Plan.md`; `docs/PVP_Engine_Race.md`; `BUGFIX_VERIFICATION.md`. Use these to revise the formal documents, and verify their claims against the selected release.